

A SMALL THEME FOR MODERN SLIDES

Infrastructure & DevOps

Docker, VPS, Nginx/SSL, workers, and monitoring

Weblearns Academy
weblearns@training.local
Odoo Full-Stack Developer Program
Lecture materials / July 2026



Agenda

- 1 Containers and Compose for Odoo
- 2 Hardened VPS and Nginx reverse proxy
- 3 systemd, backups, and TLS
- 4 Workers, indexes, and observability

TOC

1 A theme for modern talks

How this deck is taught

AWESOME-STYLE FRAME

Number rails, angled title art, and section slides are built in.

METROPOLIS DEFAULTS

Progress bars, block styles, and fonts are set from theme options.

DENSE TEACHING

Two-column frames pack objectives, examples, and pitfalls.

Docker

Nginx

SSL

Workers

1 I01 – I05

Infrastructure & DevOps · teaching block 1

I01 Containers vs Virtual Machines

Objectives, key points, and lab target

Objectives

- Explain the difference between OS virtualization and process isolation.
- Choose an appropriate runtime for Odoo development and deployment.
- Identify what containers do not solve.

Key points

- Containers share the host kernel.
- VMs provide stronger OS-level separation.
- Container filesystems are disposable by design.
- Persistent data belongs in volumes or managed storage.
- Containers improve repeatability, not application design.

Lab target

Draw an architecture for local Odoo using containers and explain which files survive when the Odoo container is deleted.

I01 Containers vs Virtual Machines

Teaching notes

Containers vs Virtual Machines is infrastructure work that directly affects reliability, data safety, and developer speed. Containers package a process with its filesystem and dependencies, while virtual machines package an entire guest operating system. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Use containers to make Odoo services repeatable and portable, but still design the host, network, storage, and backup strategy deliberately. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Run Odoo, PostgreSQL, Nginx, and workers as separate services where appropriate and keep persistent data outside disposable container layers. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Also remember

- Verification must prove behavior, not only syntax. Verify isolation assumptions by restarting containers, removing...
- In production, containers vs virtual machines choices become long-lived operational contracts. Containers reduce...

I01 Containers vs Virtual Machines

Example and pitfalls

Mental model comparison

The model highlights why containers start quickly but still depend on host and storage decisions.

Container mindset

app process + image filesystem + mounted volumes + net

VM mindset

guest OS + kernel + virtual disk + virtual network + app processes

Common mistakes

- Storing database data inside a disposable container layer.
- Assuming containers are automatically secure.
- Using one giant container for every service.
- Forgetting host-level patching and monitoring.

I02 Installing Docker Compose for Odoo Workloads

Objectives, key points, and lab target

Objectives

- Install Docker Engine and Compose plugin.
- Verify versions and permissions.
- Prepare a host for repeatable compose projects.

Key points

- Use the Compose plugin command: `docker compose`.
- Membership in the docker group is root-equivalent.
- Pin host setup in documentation or automation.
- Verify daemon and compose versions.
- Keep production hosts minimal.

Lab target

Install Docker and Compose on a disposable VM, run hello-world, and document the exact version output.

I02 Installing Docker Compose for Odoo Workloads

Teaching notes

Installing Docker Compose for Odoo Workloads is infrastructure work that directly affects reliability, data safety, and developer speed. Docker Compose is the most common entry point for running Odoo, PostgreSQL, and supporting services together on a single host. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Decide whether the host is for development or production because package sources, user permissions, filesystem paths, and firewall rules differ. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Install Docker from official packages, enable the service, add only trusted operators to the docker group, and verify compose is available as docker compose. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Also remember

- Verification must prove behavior, not only syntax. Run hello-world, inspect docker info, and create a minimal compose...
- In production, installing docker compose for odoo workloads choices become long-lived operational contracts. A clean...

I02 Installing Docker Compose for Odoo Workloads

Example and pitfalls

Ubuntu installation outline

The commands install Docker Engine and verify that the Compose plugin is available.

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg
sudo apt-get install -y docker-ce docker-ce-cli containerd.io
docker compose version
```

Common mistakes

- Installing old docker-compose v1 and mixing command syntax.
- Adding every user to the docker group casually.
- Skipping a reboot or new login after group changes.
- Debugging compose files before verifying the daemon works.

I03 Docker CLI, Images, Containers, and Docker Hub

Objectives, key points, and lab target

Objectives

- Use core Docker CLI commands confidently.
- Explain images, containers, tags, and registries.
- Inspect running Odoo services.

Key points

- Images are immutable templates.
- Containers are running or stopped instances.
- Registries store and distribute images.
- Tags are names, not guarantees of immutability.
- `docker inspect` is essential for debugging.

Lab target

Run an Odoo container, inspect its environment and mounts, then explain the difference between stopping and removing it.

I03 Docker CLI, Images, Containers, and Docker Hub

Teaching notes

Docker CLI, Images, Containers, and Docker Hub is infrastructure work that directly affects reliability, data safety, and developer speed. The Docker CLI is the operational vocabulary for seeing what is running, what image it came from, and where logs are produced. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Use tags deliberately: latest is convenient for experiments but too ambiguous for production upgrade control. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Pull images, run containers, inspect metadata, read logs, execute shells, and remove unused resources with clear intent. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Also remember

- Verification must prove behavior, not only syntax. Verify the image digest, environment, mounts, and network settings...
- In production, docker cli, images, containers, and docker hub choices become long-lived operational contracts....

I03 Docker CLI, Images, Containers, and Docker Hub

Example and pitfalls

Useful Docker inspection commands

These commands show running services, images, recent logs, interactive access, mounts, and disk usage.

```
docker ps
docker images
docker logs --tail=100 odoo-web
docker exec -it odoo-web bash
docker inspect odoo-web --format '{{json .Mounts}}'
docker system df
```

Common mistakes

- Using latest tags in production.
- Removing volumes while trying to clean images.
- Ignoring image provenance.
- Debugging from assumptions instead of docker inspect output.

I04 Dockerfile for Odoo

Objectives, key points, and lab target

Objectives

- Build a custom Odoo image.
- Install Python and system dependencies predictably.
- Keep addons and requirements layered for cache efficiency.

Key points

- Images should include code and dependencies, not secrets.
- Layer order affects build cache.
- Use requirements.txt for Python dependencies.
- Keep runtime config outside the image.
- Build and test images in CI.

Lab target

Create a Dockerfile for custom addons, build it locally, and run Odoo from the image without mounting addon code.

I04 Dockerfile for Odoo

Teaching notes

Dockerfile for Odoo is infrastructure work that directly affects reliability, data safety, and developer speed. A custom Dockerfile turns an Odoo deployment from a mutable server into a versioned artifact. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Decide which dependencies belong in the image and which configuration belongs at runtime through environment or mounted files. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Start from a trusted Odoo base image, copy requirements before addons for cache efficiency, install packages as root, and run Odoo as the intended user. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Also remember

- Verification must prove behavior, not only syntax. Build the image from a clean context and run it without bind mounts...
- In production, dockerfile for odoo choices become long-lived operational contracts. A production image should be...

I04 Dockerfile for Odoo

Example and pitfalls

Custom Odoo image

The image installs Python dependencies first and then copies custom addons with the correct ownership.

```
FROM odoo:18.0

USER root
COPY requirements.txt /tmp/requirements.txt
RUN pip3 install --no-cache-dir -r /tmp/requirements.txt

USER odoo
COPY --chown=odoo:odoo ./addons /mnt/extra-addons
ENV ODOO_RC=/etc/odoo/odoo.conf
```

Common mistakes

- Copying secrets into the image.
- Installing dependencies interactively inside a running container.
- Invalidating the build cache by copying the whole repository too early.
- Running the final container as root without need.

I05 Volumes for PostgreSQL, Filestore, and Addons

Objectives, key points, and lab target

Objectives

- Map persistent Odoo data to volumes.
- Separate database, filestore, and code storage.
- Avoid accidental data loss during container replacement.

Key points

- PostgreSQL data and Odoo filestore are both persistent.
- Named volumes are managed by Docker.
- Bind mounts expose host paths explicitly.
- Container layers are disposable.
- Backups must include database and filestore together.

Lab target

Attach a PDF to an Odoo record, recreate the Odoo container, and confirm the attachment still opens.

I05 Volumes for PostgreSQL, Filestore, and Addons

Teaching notes

Volumes for PostgreSQL, Filestore, and Addons is infrastructure work that directly affects reliability, data safety, and developer speed. Odoo persistence is split between PostgreSQL rows and the filestore, and both must be backed up together. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Design storage paths so database data, Odoo filestore, configuration, logs, and custom addons have clear ownership and backup policy. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Use named volumes or host bind mounts intentionally, and never rely on the writable container layer for anything important. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Also remember

- Verification must prove behavior, not only syntax. Verify by deleting and recreating the application container while...
- In production, volumes for postgresql, filestore, and addons choices become long-lived operational contracts....

I05 Volumes for PostgreSQL, Filestore, and Addons

Example and pitfalls

Compose volumes for Odoo data

The database and filestore use persistent volumes while custom addons are mounted read-only.

```
services:
  db:
    image: postgres:16
    volumes:
      - pgdata:/var/lib/postgresql/data
  odoo:
    image: odoo:18.0
    volumes:
      - odoo-data:/var/lib/odoo
      - ./addons:/mnt/extra-addons:ro
# ...
```

Common mistakes

- Backing up only the database and forgetting attachments.
- Mounting writable addon code in production.
- Using anonymous volumes without knowing their names.
- Deleting volumes during image cleanup.

2 I06 – I10

Infrastructure & DevOps · teaching block 2

I06 Docker Networking for Odoo

Objectives, key points, and lab target

Objectives

- Explain service discovery in Docker Compose.
- Connect Odoo to PostgreSQL by service name.
- Expose only required ports to the host.

Key points

- Compose services share a project network by default.
- Service names resolve through Docker DNS.
- ports publishes to the host; expose documents internal ports.
- Do not depend on container IP addresses.
- Limit public exposure.

Lab target

Create a compose network where only Odoo is published, then prove PostgreSQL is reachable from Odoo but not from the host port list.

I06 Docker Networking for Odoo

Teaching notes

Docker Networking for Odoo is infrastructure work that directly affects reliability, data safety, and developer speed. Compose networking gives services DNS names and private connectivity without exposing every port to the outside world. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Decide which services need host access, which only need internal network access, and which should be isolated from public traffic. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Use service names such as db as hostnames, publish only Nginx or Odoo ports needed by the environment, and avoid hard-coded container IP addresses. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Also remember

- Verification must prove behavior, not only syntax. Verify connectivity with `docker exec`, `getent hosts`, and application...
- In production, docker networking for odoo choices become long-lived operational contracts. A clean network design...

I06 Docker Networking for Odoo

Example and pitfalls

Internal database network

Odoo connects to PostgreSQL using the db service name without publishing PostgreSQL to the host.

```
services:
  odoo:
    image: odoo:18.0
    environment:
      HOST: db
      USER: odoo
      PASSWORD: odoo
    ports:
      - "8069:8069"
    depends_on:
# ...
```

Common mistakes

- Publishing PostgreSQL publicly without a reason.
- Using container IPs that change after recreation.
- Assuming depends_on means the database is ready for connections.
- Mixing unrelated stacks on the same exposed ports.

I07 Docker Compose Fundamentals

Objectives, key points, and lab target

Objectives

- Read and write compose service definitions.
- Use environment, volumes, ports, and depends_on.
- Operate a compose project safely.

Key points

- docker compose config validates and expands configuration.
- Services are the main units.
- Environment can come from .env and env_file.
- Restart policy controls daemon behavior after failure.
- Compose project names affect resource names.

Lab target

Write a minimal compose file with Odoo and PostgreSQL, run docker compose config, and explain every generated service field.

I07 Docker Compose Fundamentals

Teaching notes

Docker Compose Fundamentals is infrastructure work that directly affects reliability, data safety, and developer speed. Compose turns a multi-command deployment into a declarative project that can be started, stopped, inspected, and versioned. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Design compose files so the default command is safe for the target environment and overrides are explicit. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Define services, images or build contexts, environment variables, volumes, networks, healthchecks, and restart policies where appropriate. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Also remember

- Verification must prove behavior, not only syntax. Verify with config rendering, logs, service status, and restart...
- In production, docker compose fundamentals choices become long-lived operational contracts. Production compose usage...

I07 Docker Compose Fundamentals

Example and pitfalls

Common compose operations

These commands validate, start, inspect, restart, and stop a compose project.

```
docker compose config
docker compose up -d
docker compose logs -f --tail=100 odoo
docker compose ps
docker compose restart odoo
docker compose down
```

Common mistakes

- Editing running containers instead of compose files.
- Skipping docker compose config before deployment.
- Confusing down with volume removal.
- Letting project names change between runs unexpectedly.

I08 Multi-Container Odoo and PostgreSQL

Objectives, key points, and lab target

Objectives

- Run Odoo and PostgreSQL as separate services.
- Configure database credentials and health checks.
- Understand startup ordering and readiness.

Key points

- Odoo and PostgreSQL should be separate services.
- `depends_on` controls order, not readiness.
- Healthchecks make service state visible.
- Use persistent PostgreSQL storage.
- Restart behavior should be intentional.

Lab target

Run the two-container stack, restart PostgreSQL, and capture the relevant Odoo logs during reconnect.

I08 Multi-Container Odoo and PostgreSQL

Teaching notes

Multi-Container Odoo and PostgreSQL is infrastructure work that directly affects reliability, data safety, and developer speed. Separating Odoo and PostgreSQL mirrors the real dependency between an application server and a database server. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Choose explicit database credentials, persistent storage, healthchecks, and restart behavior before adding custom addons or proxy layers. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Use environment variables or config files for `db_host`, `db_user`, and `db_password`, and define a PostgreSQL healthcheck because `depends_on` is not readiness. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Also remember

- Verification must prove behavior, not only syntax. Verify by restarting only the database, then only Odoo, and...
- In production, multi-container odoo and postgresql choices become long-lived operational contracts. A stable...

I08 Multi-Container Odoo and PostgreSQL

Example and pitfalls

Odoo and PostgreSQL compose stack

The stack separates services and exposes database readiness through a healthcheck.

```
services:
  db:
    image: postgres:16
    restart: unless-stopped
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: postgres
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
# ...
```

Common mistakes

- Putting PostgreSQL inside the Odoo container.
- Using default passwords in shared environments.
- Assuming startup order means readiness.
- Restarting the whole stack for a single-service issue.

I09 .env Files and Runtime Configuration

Objectives, key points, and lab target

Objectives

- Use .env files for non-secret and secret runtime values carefully.
- Understand Compose variable substitution.
- Separate committed examples from real credentials.

Key points

- .env supports Compose variable substitution.
- Commit examples, not real secrets.
- docker compose config shows resolved values.
- Use clear variable names.
- Production secrets may need a stronger mechanism.

Lab target

Create .env.example and .gitignore entries for a compose project, then render the final compose config with local values.

I09 .env Files and Runtime Configuration

Teaching notes

.env Files and Runtime Configuration is infrastructure work that directly affects reliability, data safety, and developer speed. .env files make compose projects easier to configure, but they also become a common source of leaked secrets. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Decide which variables are safe defaults, which are environment-specific, and which should come from a secret manager in production. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Commit .env.example, ignore real .env files, and use explicit variable names for database credentials, domain names, image tags, and worker settings. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Also remember

- Verification must prove behavior, not only syntax. Verify the rendered configuration with `docker compose config` and...
- In production, .env files and runtime configuration choices become long-lived operational contracts. A disciplined...

I09 .env Files and Runtime Configuration

Example and pitfalls

.env.example

The example documents required settings without pretending that the sample password is production-ready.

```
ODOO_IMAGE=odoo:18.0
POSTGRES_USER=odoo
POSTGRES_PASSWORD=replace-in-real-env
ODOO_DB_HOST=db
ODOO_WORKERS=4
ODOO_LIMIT_MEMORY_HARD=2684354560
DOMAIN=odoo.example.com
```

Common mistakes

- Committing real .env files.
- Assuming .env variables are automatically available inside containers.
- Using vague names such as PASSWORD for multiple services.
- Skipping rendered config review before deployment.

I10 Upgrading Containers Safely

Objectives, key points, and lab target

Objectives

- Plan image and module upgrades.
- Back up database and filestore before changes.
- Roll forward or roll back from a known state.

Key points

- Pin image tags before upgrading.
- Back up database and filestore together.
- Test migrations on a restored copy.
- Keep previous image available for rollback.
- Review logs before declaring success.

Lab target

Write a one-page upgrade runbook for an Odoo container stack, including rollback criteria and validation checks.

I10 Upgrading Containers Safely

Teaching notes

Upgrading Containers Safely is infrastructure work that directly affects reliability, data safety, and developer speed. Container upgrades are easy to start and expensive to recover from if database migrations and filestore compatibility are ignored. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Separate base image upgrades, custom addon changes, and database migrations into a planned sequence with a tested backup. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Pull or build the new image, stop traffic, take backups, run Odoo update commands on staging first, and keep the previous image tag available. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Also remember

- Verification must prove behavior, not only syntax. Verify module updates, logs, scheduled jobs, reports, and core user...
- In production, upgrading containers safely choices become long-lived operational contracts. Safe upgrades are boring...

I10 Upgrading Containers Safely

Example and pitfalls

Upgrade command sequence

The sequence backs up before starting the new image and runs an explicit module update.

```
docker compose pull odoo
docker compose stop odoo
./scripts/backup_odoo.sh
docker compose up -d odoo
docker compose exec odoo odoo -d prod -u training --stop-after-init
docker compose logs --tail=200 odoo
```

Common mistakes

- Using latest tags and not knowing what changed.
- Upgrading production before staging.
- Backing up the database without the filestore.
- Skipping post-upgrade functional checks.

3 I11 – I15

Infrastructure & DevOps · teaching block 3

I11 CI/CD with GitHub Actions Basics

Objectives, key points, and lab target

Objectives

- Create a basic CI workflow for Odoo modules.
- Run linting and tests on pull requests.
- Build container images from commits.

Key points

- Run checks on pull_request and push.
- Use PostgreSQL services for integration tests.
- Cache dependencies cautiously.
- Protect secrets from untrusted forks.
- Keep workflows readable and incremental.

Lab target

Create a CI workflow that compiles custom addon Python files and intentionally break one file to prove the check fails.

I11 CI/CD with GitHub Actions Basics

Teaching notes

CI/CD with GitHub Actions Basics is infrastructure work that directly affects reliability, data safety, and developer speed. CI/CD turns local confidence into repeatable checks that run before code reaches shared environments. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Start with fast checks that catch common failures: syntax, manifest validity, unit tests, and image build problems. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Use GitHub Actions workflows with pinned action versions, service containers for PostgreSQL where needed, and artifacts for logs or coverage. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Also remember

- Verification must prove behavior, not only syntax. Verify that a failing test fails the workflow and that secrets are...
- In production, ci/cd with github actions basics choices become long-lived operational contracts. Production deployment...

I11 CI/CD with GitHub Actions Basics

Example and pitfalls

Basic GitHub Actions workflow

The workflow starts with lightweight checks that catch syntax and manifest problems.

```
name: odoo-ci

on:
  pull_request:
  push:
    branches: [main]

jobs:
  python-checks:
    runs-on: ubuntu-latest
  # ...
```

Common mistakes

- Putting deployment secrets in pull_request workflows from forks.
- Running huge end-to-end jobs before cheap syntax checks.
- Ignoring workflow failure logs.
- Letting CI use a different dependency set than production.

I12 VPS Provisioning and Production Directory Layout

Objectives, key points, and lab target

Objectives

- Prepare a VPS for Odoo deployment.
- Design a clear production directory structure.
- Separate app code, config, backups, logs, and secrets.

Key points

- Use a non-root deploy or odoo user.
- Keep configuration and secrets separate from code.
- Define backup and log paths explicitly.
- Document the directory layout.
- Provision disk and memory for growth.

Lab target

Provision a small test VPS or VM and create the directory layout with ownership and permissions suitable for an odoo deploy user.

I12 VPS Provisioning and Production Directory Layout

Teaching notes

VPS Provisioning and Production Directory Layout is infrastructure work that directly affects reliability, data safety, and developer speed. A VPS is simple enough for small Odoo deployments but still needs disciplined layout and host preparation. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Provision with a supported OS, enough memory and disk IOPS, a non-root deploy user, firewall rules, time sync, and monitoring access. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Create predictable directories such as `/opt/odoo/app`, `/opt/odoo/config`, `/opt/odoo/backups`, `/var/log/odoo`, and `/srv/odoo/filestore` depending on the deployment style. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Also remember

- Verification must prove behavior, not only syntax. Verify permissions by running the service as the target user and...
- In production, vps provisioning and production directory layout choices become long-lived operational contracts. A...

I12 VPS Provisioning and Production Directory Layout

Example and pitfalls

Production layout example

The layout keeps operational assets separate and easy to back up or audit.

```
/opt/odoo/  
  app/           # compose file or deployment repository  
  config/        # odoo.conf, nginx snippets, etc.  
  backups/       # database and filestore backups  
  scripts/       # backup, restore, health checks  
/var/log/odoo/   # application logs  
/srv/odoo/filestore/ # optional external filestore path
```

Common mistakes

- Running everything from a developer's home directory.
- Mixing backups into the application repository.
- Leaving important files owned by root after manual commands.
- Not documenting which path is authoritative.

I13 SSH Hardening

Objectives, key points, and lab target

Objectives

- Reduce common SSH attack surface.
- Use key-based authentication.
- Protect administrative access without locking out operators.

Key points

- Use public key authentication.
- Disable direct root login.
- Keep an active session while testing changes.
- Limit sudo and user access.
- Maintain an emergency recovery path.

Lab target

Harden SSH on a disposable VM, prove key login works in a second session, and document the rollback path.

I13 SSH Hardening

Teaching notes

SSH Hardening is infrastructure work that directly affects reliability, data safety, and developer speed. SSH is usually the front door to a VPS, so weak SSH posture undermines every application hardening effort. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Design access around named users, public keys, sudo policy, and an emergency recovery path from the VPS provider console. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure. Implementation should be explicit and repeatable. Disable password login, disable root login, restrict users where practical, install fail2ban or equivalent protection, and keep authorized keys reviewed. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Also remember

- Verification must prove behavior, not only syntax. Verify a second session can connect before closing the first and...
- In production, ssh hardening choices become long-lived operational contracts. Good SSH hardening balances security and...

I13 SSH Hardening

Example and pitfalls

sshd hardening fragment

The fragment disables password and root login while allowing a named deploy user.

```
# /etc/ssh/sshd_config.d/10-hardening.conf
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
AllowUsers deploy
ClientAliveInterval 300
ClientAliveCountMax 2
```

Common mistakes

- Restarting SSH after changes without an open backup session.
- Sharing one private key between operators.
- Disabling root login before creating a sudo-capable user.
- Forgetting provider console recovery.

I14 Nginx Reverse Proxy for Odoo

Objectives, key points, and lab target

Objectives

- Proxy public HTTP traffic to Odoo.
- Set headers required for correct URL generation.
- Configure timeouts for long Odoo requests.

Key points

- Enable `proxy_mode` in Odoo when behind a proxy.
- Forward Host and X-Forwarded headers.
- Set body size for imports and attachments.
- Tune timeouts for reports and upgrades.
- Keep upstream ports private when possible.

Lab target

Put Nginx in front of Odoo, enable `proxy_mode`, and confirm generated `web.base.url` uses the public hostname.

I14 Nginx Reverse Proxy for Odoo

Teaching notes

Nginx Reverse Proxy for Odoo is infrastructure work that directly affects reliability, data safety, and developer speed. Nginx usually terminates public HTTP traffic and forwards application requests to Odoo on an internal port. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Decide where TLS terminates, which hostnames are valid, and whether Odoo runs in `proxy_mode` so it trusts forwarded headers correctly. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Configure `proxy_pass`, Host, X-Forwarded headers, body size, buffering, and timeouts appropriate for Odoo imports and reports. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Also remember

- Verification must prove behavior, not only syntax. Verify real client IPs, generated URLs, file uploads, long report...
- In production, nginx reverse proxy for odoo choices become long-lived operational contracts. A correct reverse proxy...

I14 Nginx Reverse Proxy for Odoo

Example and pitfalls

Nginx Odoo proxy server

The proxy forwards standard headers and allows long Odoo requests to complete.

```
server {  
    listen 80;  
    server_name odoo.example.com;  
  
    client_max_body_size 100m;  
  
    location / {  
        proxy_pass http://127.0.0.1:8069;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
# ...
```

Common mistakes

- Forgetting `proxy_mode` in Odoo configuration.
- Leaving Odoo publicly exposed when Nginx should be the entry point.
- Using low timeouts that break large reports or imports.
- Not forwarding X-Forwarded-Proto, causing bad redirects.

I15 Ports 80 and 443 to 8069, Longpolling 8072, and Let's Encrypt

Objectives, key points, and lab target

Objectives

- Route public HTTP and HTTPS traffic to Odoo.
- Handle longpolling or websocket-style notification endpoints.
- Install TLS certificates with Certbot.

Key points

- Public users should use 80 and 443.
- Odoo backend often listens on 8069 internally.
- Longpolling commonly uses 8072 in worker deployments.
- Certbot can automate Let's Encrypt certificates.
- Test certificate renewal before relying on it.

Lab target

Configure HTTPS for a test hostname, add a longpolling location, and run `certbot renew --dry-run`.

I15 Ports 80 and 443 to 8069, Longpolling 8072, and Let's Encrypt

Teaching notes

Ports 80 and 443 to 8069, Longpolling 8072, and Let's Encrypt is infrastructure work that directly affects reliability, data safety, and developer speed. Users should reach Odoo on standard web ports while the application listens internally on ports such as 8069 and longpolling on 8072. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Plan firewall rules, Nginx server blocks, TLS termination, renewal automation, and the route for bus or longpolling traffic. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Use Certbot for Let's Encrypt certificates, redirect HTTP to HTTPS, and add a separate location for longpolling when the deployment uses a dedicated port. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Also remember

- Verification must prove behavior, not only syntax. Verify certificate renewal, live chat or bus notifications,...
- In production, ports 80 and 443 to 8069, longpolling 8072, and let's encrypt choices become long-lived operational...

I15 Ports 80 and 443 to 8069, Longpolling 8072, and Let's Encrypt

Example and pitfalls

Nginx HTTPS and longpolling locations

The server routes normal requests to 8069 and longpolling requests to 8072 under HTTPS.

```
server {  
    listen 443 ssl http2;  
    server_name odoo.example.com;  
  
    ssl_certificate /etc/letsencrypt/live/odoo.example.com/fullchain.pem;  
    ssl_certificate_key /etc/letsencrypt/live/odoo.example.com/privkey.pem;  
  
    location /longpolling/ {  
        proxy_pass http://127.0.0.1:8072;  
        proxy_set_header Host $host;  
  
    # ...  
}
```

Common mistakes

- Opening 8069 and 8072 publicly when only Nginx should be exposed.
- Forgetting automatic certificate renewal.
- Routing longpolling to the wrong upstream.
- Not testing real-time notifications after enabling workers.

4 I16 – I20

Infrastructure & DevOps · teaching block 4

I16 systemd odoo.service

Objectives, key points, and lab target

Objectives

- Run Odoo as a systemd service.
- Define restart behavior and service user.
- Use journalctl for logs and diagnosis.

Key points

- Run Odoo under a dedicated user.
- Keep ExecStart explicit.
- Use systemctl enable for boot startup.
- Use journalctl -u for logs.
- Restart policy should match operational expectations.

Lab target

Create a systemd unit on a test VM, start it, force a failure, and confirm restart behavior in journalctl.

I16 systemd odoo.service

Teaching notes

systemd odoo.service is infrastructure work that directly affects reliability, data safety, and developer speed. Some deployments run Odoo directly on a host instead of containers, and systemd is the standard process supervisor for that model. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Decide the service user, config file path, Python environment, addons path, restart policy, and log destination before writing the unit. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Create a unit file with ExecStart, User, WorkingDirectory, Restart, and dependency on PostgreSQL or network availability as appropriate. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Also remember

- Verification must prove behavior, not only syntax. Verify daemon reload, enable, start, restart, failure restart, and...
- In production, systemd odoo.service choices become long-lived operational contracts. A clear systemd unit makes a...

I16 systemd odoo.service

Example and pitfalls

Odoo systemd unit

The unit runs Odoo from a virtual environment with a dedicated user and restarts after failures.

```
[Unit]
Description=Odoo Application Server
After=network.target postgresql.service

[Service]
Type=simple
User=odoo
Group=odoo
WorkingDirectory=/opt/odoo/server
ExecStart=/opt/odoo/venv/bin/python3 /opt/odoo/server/odoo-bin -c /etc/odoo/odoo.conf
# ...
```

Common mistakes

- Running Odoo manually in screen or tmux as the production supervisor.
- Starting the service as root without need.
- Forgetting systemctl daemon-reload after editing the unit.
- Hiding logs in an unmanaged file path.

I17 Backup Cron for Database and Filestore

Objectives, key points, and lab target

Objectives

- Back up PostgreSQL and Odoo filestore together.
- Schedule backups with cron.
- Test restore, retention, and failure alerts.

Key points

- Database and filestore must be backed up as a pair.
- Timestamp backup names.
- Store copies off the application host.
- Test restore regularly.
- Alert on backup failures.

Lab target

Create a backup script and cron entry, then restore the output to a separate test database and open an attachment.

I17 Backup Cron for Database and Filestore

Teaching notes

Backup Cron for Database and Filestore is infrastructure work that directly affects reliability, data safety, and developer speed. Backups are only useful when they can be restored, and Odoo requires both database and filestore for a complete restore. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Design backup frequency, retention, encryption, offsite storage, and restore objectives before writing the cron line. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Use `pg_dump` or `pg_dumpall` as appropriate, archive the filestore matching the database, name backups with timestamps, and prune old files safely. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Also remember

- Verification must prove behavior, not only syntax. Verify by restoring to a separate database and opening attachments,...
- In production, backup cron for database and filestore choices become long-lived operational contracts. Production...

I17 Backup Cron for Database and Filestore

Example and pitfalls

Backup script fragment

The script writes a timestamped database dump and matching filestore archive, then prunes old local backups.

```
#!/usr/bin/env bash
set -euo pipefail

DB=prod
STAMP=$(date +%Y%m%d-%H%M%S)
DEST=/opt/odoo/backups/$STAMP
mkdir -p "$DEST"

pg_dump -Fc "$DB" > "$DEST/$DB.dump"
tar -C /var/lib/odoo/filestore -czf "$DEST/filestore.tar.gz" "$DB"
# ...
```

Common mistakes

- Backing up only PostgreSQL.
- Never testing restore.
- Keeping all backups on the same disk as production.
- Writing cron jobs that fail silently.

I18 Odoo Workers Architecture and Worker Calculation

Objectives, key points, and lab target

Objectives

- Explain threaded versus multi-worker Odoo modes.
- Calculate an initial worker count.
- Reserve capacity for cron and longpolling.

Key points

- workers > 0 enables multi-process mode.
- A common starting formula is CPU cores * 2 + 1, adjusted for memory.
- Cron workers consume capacity.
- Longpolling or bus traffic may need separate handling.
- Measure before and after changes.

Lab target

Calculate initial workers for a 4-core, 8 GB VPS running Odoo and PostgreSQL, then justify memory reserves.

I18 Odoo Workers Architecture and Worker Calculation

Teaching notes

Odoo Workers Architecture and Worker Calculation is infrastructure work that directly affects reliability, data safety, and developer speed. Odoo workers let the application handle concurrent HTTP requests with multiple processes instead of one threaded development-style server. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Base the initial worker count on CPU cores, memory, traffic profile, cron load, and whether the host also runs PostgreSQL. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Set `workers`, `max_cron_threads`, `proxy_mode`, and `gevent` or `longpolling` configuration according to the Odoo version and deployment model. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Also remember

- Verification must prove behavior, not only syntax. Verify under load with realistic requests, not only by checking...
- In production, odoo workers architecture and worker calculation choices become long-lived operational contracts....

I18 Odoo Workers Architecture and Worker Calculation

Example and pitfalls

odoo.conf worker settings

The settings enable worker mode with conservative time and memory limits for a modest host.

```
[options]
workers = 5
max_cron_threads = 1
proxy_mode = True
limit_time_cpu = 120
limit_time_real = 240
limit_memory_soft = 2147483648
limit_memory_hard = 2684354560
```

Common mistakes

- Using the CPU formula without checking memory.
- Forgetting cron capacity.
- Running many workers against too few PostgreSQL connections.
- Changing workers without load testing.

I19 Memory Limits, Time Limits, PostgreSQL Pooling, and max_connections

Objectives, key points, and lab target

Objectives

- Configure Odoo request memory and time guards.
- Relate Odoo workers to PostgreSQL connections.
- Plan pooling or max_connections changes safely.

Key points

- Odoo workers consume memory independently.
- db_maxconn controls Odoo's database connection pool per process.
- PostgreSQL max_connections must cover all clients.
- Memory limits recycle runaway workers.
- Long reports may need time limit review.

Lab target

Create a connection and memory budget for an 8-worker Odoo deployment and propose safe PostgreSQL max_connections.

I19 Memory Limits, Time Limits, PostgreSQL Pooling, and `max_connections`

Teaching notes

Memory Limits, Time Limits, PostgreSQL Pooling, and max_connections is infrastructure work that directly affects reliability, data safety, and developer speed. Odoo and PostgreSQL capacity problems often appear together because more workers can mean more database connections and more memory pressure. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Design limits from the host resources, request profile, scheduled jobs, and PostgreSQL memory settings instead of copying values from another deployment. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Set limit_memory_soft, limit_memory_hard, limit_time_cpu, limit_time_real, db_maxconn, and PostgreSQL max_connections with enough headroom for maintenance sessions. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Also remember

- Verification must prove behavior, not only syntax. Verify by monitoring worker recycling, slow requests, database...
- In production, memory limits, time limits, postgresql pooling, and max_connections choices become long-lived...

I19 Memory Limits, Time Limits, PostgreSQL Pooling, and max_connections

Example and pitfalls

Connection budget example

The calculation reminds operators to budget for all database clients, not only HTTP workers.

```
# Example budget
workers = 5
max_cron_threads = 1
db_maxconn = 16
```

```
# Potential Odoo connections: (workers + cron + main process) * db_maxconn
# Add admin tools, monitoring, backup jobs, and migration sessions before
# setting PostgreSQL max_connections.
```

Common mistakes

- Increasing workers without increasing database capacity.
- Setting max_connections high without enough PostgreSQL memory.
- Disabling time limits to hide slow code.
- Ignoring cron and migration connections.

I20 Indexes, pg_stat_statements, Redis Sessions, Load Balancing, and Prometheus Grafana

Objectives, key points, and lab target

Objectives

- Identify slow SQL with pg_stat_statements.
- Explain when missing indexes hurt Odoo.
- Describe Redis sessions, load balancing, and monitoring basics.

Key points

- pg_stat_statements shows expensive query patterns.
- Indexes should match frequent domains and joins.
- Redis sessions can help multi-node session consistency.
- Load balancing requires shared filestore and session strategy.
- Prometheus and Grafana turn metrics into alerts and dashboards.

Lab target

Enable pg_stat_statements on a staging database, identify one slow query pattern, and propose a measured index or code change.

I20 Indexes, pg_stat_statements, Redis Sessions, Load Balancing, and Prometheus Grafana

Teaching notes

Indexes, `pg_stat_statements`, Redis Sessions, Load Balancing, and Prometheus Grafana is infrastructure work that directly affects reliability, data safety, and developer speed. At scale, Odoo performance is usually constrained by database queries, worker saturation, cache behavior, or lack of visibility. Senior engineers treat these topics as part of the product because users experience slow pages, broken backups, and unsafe upgrades as product failures.

The design decision starts by separating local development, staging, and production needs. Design observability before firefighting: know which metrics, logs, and query statistics reveal the difference between slow Python, slow SQL, and overloaded infrastructure. The right architecture is the simplest one that can be restored, upgraded, monitored, and explained under incident pressure.

Implementation should be explicit and repeatable. Enable `pg_stat_statements`, inspect high-total-time queries, add indexes only after understanding domains, and consider Redis-backed sessions or sticky load balancing for multi-node deployments. Prefer configuration files committed to the repository, secrets supplied by the environment, and operational commands that can be run by another engineer.

Also remember

- Verification must prove behavior, not only syntax. Verify every performance change with before-and-after measurements,...
- In production, indexes, `pg_stat_statements`, redis sessions, load balancing, and prometheus grafana choices become...

I20 Indexes, pg_stat_statements, Redis Sessions, Load Balancing, and Prometheus Grafana

Example and pitfalls

Inspect slow queries

The query identifies expensive SQL patterns; the example index matches a common state and date domain.

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

SELECT calls,
       total_exec_time,
       mean_exec_time,
       rows,
       query
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;
# ...
```

Common mistakes

- Adding indexes blindly for every slow query.
- Load balancing without shared filestore or session planning.
- Creating dashboards with no alert thresholds.
- Ignoring write overhead from extra indexes.

I20 Indexes, pg_stat_statements, Redis Sessions, Load Balancing, and Prometheus Grafana

Extra patterns

Observability layers

-

Σ You should now be able to...

Section wrap-up

- I01 Containers vs Virtual Machines
- I02 Installing Docker Compose for Odoo Workloads
- I03 Docker CLI, Images, Containers, and Docker Hub
- I04 Dockerfile for Odoo
- I05 Volumes for PostgreSQL, Filestore, and Addons
- I06 Docker Networking for Odoo
- I07 Docker Compose Fundamentals
- I08 Multi-Container Odoo and PostgreSQL
- Complete every lab before the next section
- Keep a command/error journal
- Prefer disposable databases for experiments
- Re-read pitfalls before production changes

Docker

Nginx

SSL

Workers

NEXT STEP

Complete the section labs

Open the next presentation deck

Section 07 · Infrastructure

